

Capstone Project – OrangeHRM Enterprise Automation Testing

Prepared by: Arun N

Project Information

Field	Details
Project Type	Enterprise HR Management Automation Testing
Application	OrangeHRM (https://opensource-demo.orangehrmlive.com)
Automation Tool	Playwright
Programming Language	JavaScript
Framework Design	Page Object Model (POM)
Total Planned Test Cases	120+

Project Overview

This capstone project develops a **scalable automation testing framework** for the **OrangeHRM platform** using **Playwright with JavaScript**, structured around the **Page Object Model (POM)** to ensure readability, reusability, maintainability, and long-term stability.

The framework validates **critical enterprise workflows** including authentication, dashboard validation, employee and leave management, recruitment, user management, search and filtering, responsive UI behavior, API monitoring, regression testing, and cross-browser execution.

By emphasizing **enterprise-grade practices** such as reusable architecture, modular design, robust reporting with Playwright HTML and Allure, CI/CD integration, and debugging support, the project demonstrates a professional QA automation framework suitable for both **academic capstone requirements** and **real-world enterprise application testing**.

Why OrangeHRM Was Selected

OrangeHRM was chosen as the application under test because it is a **modern enterprise HR management platform** with dynamic UI components, responsive layouts, and realistic business workflows — making it highly suitable for **enterprise-level automation testing**.

It offers automation opportunities across **authentication, dashboard workflows, employee and leave management, recruitment operations, user management, search and filtering, responsive UI validation, cross-browser execution, and API/network monitoring**.

OrangeHRM also reflects real-world enterprise challenges such as **asynchronous page loading, dynamic tables, session handling, role-based workflows, responsive layout behavior, and multi-browser compatibility**. To ensure stable and repeatable automation, the project focuses on **safe enterprise workflows** while avoiding sensitive production-level configurations and external third-party integrations.

Project Objectives

The objective of this project is to design and implement a **maintainable enterprise-level automation testing framework** for the **OrangeHRM platform** using **Playwright with JavaScript**. Built on the **Page Object Model (POM)**, the framework ensures scalability, reusability, and long-term stability of automation scripts.

The testing plan validates **critical enterprise workflows** including authentication, dashboard validation, employee and leave management, recruitment operations, user management, search and filtering, responsive UI behavior, API monitoring, and cross-browser execution.

In addition, the project emphasizes **reusable architecture, stable automation execution, enterprise QA practices, reporting and debugging support, data-driven testing, and scalable POM implementation**, ensuring alignment with both **academic capstone requirements** and **professional QA standards**.

Technology Stack

Category	Tools / Technologies
Automation Framework	Playwright
Language	JavaScript
Runtime Environment	Node.js
Design Pattern	Page Object Model (POM)
IDE	Visual Studio Code
Reporting	HTML Report & Allure Report
Version Control	Git & GitHub
CI/CD	GitHub Actions

Scope of Testing

Category	Description
In Scope	Authentication validation, dashboard workflows, employee management, leave management, recruitment operations, search and filtering validation, responsive UI checks, API monitoring, and cross-browser execution
Out of Scope	Backend database validation, production configuration changes, external third-party integrations, and manipulation of sensitive organizational data
Safe Boundary	Tests validate enterprise workflows within safe operational limits without affecting sensitive configurations or business-critical data

Services & Test Case Breakdown (8 × 15 = 120+ Test Cases)

No.	Module / Service	Coverage Focus	Count	Severity
1	Login & Authentication Validation	Login workflows, session handling, access validation, and authentication behavior	15	Critical
2	Dashboard Validation	Dashboard widgets, menus, shortcuts, navigation flow, and visibility validation	15	High
3	Employee Management (PIM)	Product search, category browsing, filters, sorting, pagination, and catalogue validation.	15	Critical
4	Leave Management Validation	Leave application, approval workflows, leave balance, and leave tracking validation	15	Critical
5	Recruitment Workflow Validation	Candidate management, recruitment forms, resume upload, and recruitment validation	15	High
6	Admin & User Management	User roles, permissions, user search, filters, and management workflows	15	Critical
7	Search, Filters & Table Validation	Search operations, filters, sorting, pagination, and enterprise table validation	15	High
8	API, Responsive & Cross-Browser Validation	API/network validation, responsive layouts, viewport validation, and multi-browser execution	15	High

The planned 120+ test cases ensure comprehensive coverage across critical business workflows and validation scenarios.

Detailed Module Coverage

M1 – Login & Authentication Validation

Validate login functionality, invalid credential handling, session persistence, logout behavior, access restrictions, and authentication-related UI validations.

M2 – Dashboard Validation

Validate dashboard widgets, menus, shortcuts, navigation flow, notification visibility, sidebar behavior, and overall dashboard responsiveness.

M3 – Employee Management (PIM)

Validate employee creation, employee search, edit operations, form validation, table validation, pagination, and employee profile consistency.

M4 – Leave Management Validation

Validate leave application workflows, leave balance visibility, leave history, approval/rejection workflows, leave filters, and leave status handling.

M5 – Recruitment Workflow Validation

Validate candidate creation, resume upload functionality, recruitment filters, candidate status handling, and recruitment table validations.

M6 – Admin & User Management

Validate user creation, role assignment, permission validation, user search, role filters, table sorting, and user management workflows.

M7 – Search, Filters & Table Validation

Validate enterprise search operations, filter behavior, sorting logic, pagination workflows, table consistency, and result validation.

M8 – API, Responsive & Cross-Browser Validation

Validate API responses, network monitoring, responsive layouts across devices, viewport behavior, and execution consistency across Chromium, Firefox, and WebKit browsers.

Planned Testing Types

Testing Type	Purpose
Functional Testing	Validate end-to-end business workflows
Validation Testing	Validate invalid inputs and restrictions
UI Assertion Testing	Verify visible UI elements, labels, and content
Regression Testing	Ensure existing workflows remain stable after updates
Session Testing	Validate session persistence and user access behavior after refresh or navigation
API & Network Testing	Observe network calls, response status, and request validation
Responsive Testing	Validate usability across desktop, tablet, and mobile viewports
Cross-Browser Testing	Execute workflows across Chromium, Firefox, and WebKit

Framework Design & Automation Approach

The automation framework is implemented using **Playwright with JavaScript**, following the **Page Object Model (POM)** for scalability and maintainability.

Key features include:

- Reusable page classes and modular test structure
- Utility/helper functions and data-driven testing
- Screenshot and trace generation for debugging

- Cross-browser execution and API/network monitoring
- Comprehensive reporting with Playwright HTML and Allure

The framework relies on **stable Playwright locators** such as `getByRole()`, `getByText()`, `getByPlaceholder()`, and `locator()`, while avoiding fragile selectors and deeply nested XPath expressions to ensure robust and reliable automation.

Planned Project Structure

Folder / File	Purpose
pages/	Stores reusable Page Object Model (POM) classes
tests/	Contains Playwright test specification files
utils/	Reusable helper functions, waits, and utilities
test-data/	Stores reusable test data and JSON datasets
screenshots/	Stores execution screenshots and failure evidence
playwright-report/	Stores Playwright HTML execution reports
allure-results/	Stores Allure reporting results
docs/	Contains project proposal and project documentation
README.md	Project overview, setup steps, and execution guide
package.json	Manages project dependencies and scripts
playwright.config.js	Central Playwright configuration file
.gitignore	Excludes unnecessary files from Git tracking

Test Data Strategy

The framework employs **safe, configurable, and reusable test data** to ensure stable automation execution across enterprise workflows while avoiding reliance on sensitive organizational information.

Key practices include:

- Using **reusable employee, user, and candidate datasets** for validation workflows
- Maintaining **configurable test data** in separate JSON/data files
- Avoiding **hardcoded values** inside test scripts
- Leveraging **dynamic search inputs and filter values** for flexible validation
- Supporting **data-driven testing approaches** across modules

This strategy enhances **maintainability, scalability, flexibility, and execution reliability**, ensuring robust automation across diverse enterprise scenarios.

Reporting & Evidence Strategy

The framework generates **professional execution evidence** through:

- Playwright HTML and Allure reports
- Screenshots for failed scenarios
- Trace files for debugging
- GitHub Actions execution logs
- Cross-browser execution reports

These reports provide clear visibility into **pass/fail statistics, execution trends, failure details, flaky test detection, screenshot evidence, and debugging information**, ensuring both transparency and reliability in test outcomes.

Risks & Mitigation

Risk	Mitigation
Dynamic UI changes	Use stable Playwright locators and maintain selectors inside reusable page objects
Dynamic table updates	Use flexible assertions and configurable test data instead of hardcoded values
Session inconsistencies	Validate workflows using reusable session handling logic
Slow application response	Use Playwright auto-waiting and proper timeout handling
Role-based access restrictions	Validate workflows using appropriate user roles and permissions
Cross-browser inconsistencies	Execute validation across Chromium, Firefox, and WebKit to ensure consistent application behavior

Expected Deliverables

The project will deliver a **professional-grade automation framework** showcasing enterprise QA practices. Key outputs include:

- **Playwright automation framework** with organized **Page Object Model (POM)** architecture
- Automated, **reusable test suites** covering enterprise workflows
- **API and network validation** implementation
- **Cross-browser execution support**
- **Playwright HTML and Allure reports** for detailed insights
- **Screenshots and trace evidence** for debugging
- A **GitHub repository** with documentation and CI/CD integration
- **Service-wise automation coverage** demonstrating breadth and reliability

Conclusion

This capstone project demonstrates the design and implementation of a **scalable, maintainable automation testing framework** for the **OrangeHRM enterprise HR platform** using **Playwright with JavaScript**.

The framework validates **critical enterprise workflows** such as authentication, employee management, recruitment, leave workflows, responsive behavior, and cross-browser execution, while adhering to **industry-standard QA practices**.

It highlights practical **SDET skills** including framework design, reusable scripting, reporting, debugging, API validation, regression support, and enterprise automation engineering with modern Playwright technologies — showcasing both **academic rigor** and **professional relevance**.